

2、性能调优与线上问题排查

1、做过JVM调优吗？怎么做的？

分析思路：

1. 测量优于猜测

- 先观察，不盲目调参

2. 明确目标

- 吞吐量？延迟？内存？

3. 由上至下排查

操作系统 → JVM → 应用代码

实战步骤：

1、收集数据

```
# 查看GC概况
jstat -gcutil <pid> 1000

# 导出堆转储
jmap -dump:format=b,file=heap.hprof <pid>

# 打印GC日志
-XX:+PrintGCDetails -XX:+PrintGCDateStamps
```

2、分析问题：

现象	可能原因	解决方案
Minor GC频繁	新生代太小	-Xmn增大
Full GC频繁	老年代满/内存泄漏	分析堆找泄漏
OOM	内存不足/泄漏	dump分析
CPU高	GC线程/代码问题	jstack分析

3、调整参数

```
# 典型调优配置（低延迟场景）
-Xms4g -Xmx4g
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:InitiatingHeapOccupancyPercent=45
```

拓展点：AI大模型服务（如RAG系统）通常需要处理大量并发请求，调优目标是**高吞吐+低延迟**。推荐使用G1或ZGC，并开启GC日志实时监控。

2、如何排查线上OOM问题？

排查流程：

1. 立即处理

└─ 保留现场：不要立即重启！

└─ 导出堆转储：jmap -dump

└─ 重启服务恢复业务

2. 分析堆转储

└─ 使用MAT / jhat / Java VisualVM

└─ 找出最大对象

└─ 分析引用链（GC Roots → 泄漏对象）

3. 定位代码

└─ 修复内存泄漏

└─ 优化内存使用

快速定位命令

查看OOM错误日志

grep -i "OutOfMemoryError" /var/log/application.log

查看内存使用

jstat -gc <pid>

实时监控

jconsole

导出堆

jmap -dump:format=b,file=oom.hprof <pid>

常见OOM原因

类型	原因	解决方案
Java heap space	堆内存不足	-Xmx增大
<u>Metaspace</u>	加载类过多	-XX:MetaspaceSize
Direct buffer	NIO使用过多	检查 <u>ByteBuffer</u>
Request array	请求过大	限制参数
GC overhead	频繁GC	优化内存使用

3、CPU 100%怎么排查？

排查步骤

1. 定位Java进程

top -c # 找到CPU高的进程PID

```
# 2. 定位Java线程
top -Hp <pid>    # 找到CPU高的线程TID

# 3. 转换为十六进制
printf "%x\n" <TID>

# 4. 打印线程堆栈
jstack <pid> | grep <十六进制TID> -A 30

# 5. 分析代码
# 找到问题代码行
```

常见原因

- 死循环: `while(true)`无限循环
- 频繁GC: 对象创建太多, GC线程占用CPU
- 锁竞争: `synchronized`锁等待
- 正则表达式: `Pattern.compile`在循环中

4、频繁Full GC怎么排查?

排查思路

```
# 1. 查看GC日志
jstat -gcutil <pid> 1000

# 2. 分析GC日志
# -XX:+PrintGCDetails -Xloggc:gc.log
# 使用GCviewer分析
```

常见原因

- └─ 对象晋升失败 (担保失败)
 - | └─ 解决: `-XX:-HandlePromotionFailure` 或增大内存
- └─ 内存分配担保失败
 - | └─ 解决: 增大老年代
- └─ 内存碎片
 - | └─ 解决: 使用G1, 或降低CMS触发阈值
- └─ 显式调用`System.gc()`
 - | └─ 解决: `-XX:+DisableExplicitGC`

调优建议

```
# G1调优 - 减少Mixed GC
-XX:G1MixedGCLiveThresholdPercent=85
-XX:G1HeapWastePercent=5

# CMS调优 - 提前触发
-XX:CMSInitiatingOccupancyFraction=70
```

5、线上服务响应变慢，如何快速定位问题？

快速排查

1、查日志：异常/超时/gc相关

2、查资源：CPU/内存/磁盘/网络

3、查线程：jstack分析死锁/阻塞

4、查堆：jmap 分析内存

推荐使用Arthas

Arthas > dashboard

查看整体情况

Arthas > thread

查看线程

Arthas > watch

观察方法调用

Arthas > trace

追踪性能

6、ZGC和Shenandoah了解吗？为什么延迟更低？

ZGC (Z Garbage Collector)

- **设计目标：** 停顿时间 < 10ms，支持TB级堆内存
- **核心技术：**
 - **Colored Pointers (着色指针)：** 在指针上标记对象状态
 - **Load Barriers (读屏障)：** 并发标记时快速检查
 - **自愈式内存：** 并发重映射

对比：

特性	ZGC	Shenandoah	G1
停顿时间	<10ms	<10ms	可预测
内存上限	TB级	GB级	无限制
开源	OpenJDK	OpenJDK	JDK自带
JDK版本	JDK 11+	JDK 12+	JDK 9+

拓展：AI大模型服务（如基于Java的LangChain4j）处理请求时，延迟敏感**。ZGC的亚毫秒级停顿非常适合AI推理服务，可以避免GC导致的服务抖动。

7、为什么阿里等大厂都在用G1/ZGC？

大厂面临的挑战：

1. **大堆内存：** 单机256G~1T，CMS无法处理
2. **低延迟：** 要求TP999 < 100ms
3. **高吞吐：** 每秒处理数万请求

为什么选择G1/ZGC

特性	CMS	G1	ZGC
大堆	✗	✓	✓
低延迟	⚠	✓	✓
无碎片	✗	✓	✓
维护状态	废弃	主流	未来

阿里风格的配置示例：

```
# 容器环境 G1 调优
-XX:+UseG1GC
-XX:MaxGCPauseMillis=100
-XX:G1HeapRegionSize=16m
-XX:InitiatingHeapOccupancyPercent=35
-XX:G1ReservePercent=15

# 容器资源限制
# 注意：-Xms = -Xmx，禁用自适应大小
-Xms32g -Xmx32g
```

8、JVM参数怎么调优？能举例说明吗？

场景1：高吞吐场景（离线计算）

```
-Xms8g -Xmx8g
-XX:+UseParallelGC
-XX:ParallelGCThreads=8
-XX:+UseParallelOldGC
-XX:MaxGCPauseMillis=500
-XX:+UseAdaptiveSizePolicy
```

场景2：低延迟场景（在线服务）

```
-Xms4g -Xmx4g
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:G1HeapRegionSize=8m
-XX:InitiatingHeapOccupancyPercent=45
-XX:G1ReservePercent=10
```

场景3：超大内存+超低延迟（AI推理服务）